

Wirtschaftsinformatik
und Maschinelles Lernen
Stiftung Universität Hildesheim
Marienburger Platz 22
31141 Hildesheim
Prof. Dr. Dr. Lars Schmidt-Thieme
Christian Klötergens, M.Sc.

DA I Seminar Report

"Long-term Forecasting with TiDE: Time-series Dense Encoder"

Safieddin Alaa Adnan Sabealeish

1752736

(sabealeish@uni-hildesheim.de)

August 30, 2026

Abstract

Long-term time-series forecasting has recently shifted toward simpler and more computationally efficient models, as recent studies have shown that linear approaches can achieve competitive performance compared with complex Transformer-based architectures. However, linear models are often limited in their ability to capture non-linear relationships and incorporate time-series covariates. This report examines the Time-series Dense Encoder (TiDE), an MLP-based encoder-decoder framework proposed by Das et al. (2024) that aims to address these limitations while maintaining the efficiency of simpler forecasting models. The report presents the main ideas behind TiDE, including its feature projection, dense encoding, temporal decoding, and handling of covariates, and explains the theoretical motivation based on Linear Dynamical Systems. Its empirical performance is also analyzed on several widely used long-term forecasting benchmarks. The results reported in the original study show that TiDE achieves competitive or superior forecasting accuracy compared with several state-of-the-art methods, while providing substantial computational advantages, with training speeds reported to be 5–10 times faster than Transformer-based approaches. Overall, TiDE demonstrates that MLP-based architectures can provide an effective balance between forecasting accuracy, flexibility, and computational efficiency for long-term time-series forecasting.

1 Introduction

Long-term time-series forecasting aims to predict future values over extended time horizons based on previously observed data. It plays an important role in many real-world applications, including energy demand forecasting, traffic management, weather prediction, and resource planning. In these settings, accurate predictions are often required far into the future, making it important for forecasting models to capture both short-term patterns and long-term dependencies in the data.

In recent years, deep learning has become an important approach to time-series forecasting. In particular, Transformer-based models have attracted considerable attention because of their ability to model relationships between different points in a sequence using self-attention. However, the computational cost of attention increases substantially with the length of the input sequence. This can make Transformer-based approaches expensive for long-term forecasting, where relatively long historical windows may be required. Their increased complexity can also introduce unnecessary modeling capacity for some forecasting problems.

Furthermore, recent research has shown that much simpler linear models can achieve surprisingly competitive results on long-term forecasting benchmarks. Specifically, work by Zeng et al. [16] demonstrates that Transformer-based architectures may not be as effective as previously assumed for time series forecasting, often being outperformed by simple linear baselines. These findings challenge the assumption that increasingly complex architectures are always necessary to obtain accurate forecasts. Nevertheless, purely linear models have important limitations. They are generally unable to represent complex non-linear relationships and may have limited support for incorporating additional information, such as time-dependent covariates, calendar information, weather conditions, or static characteristics of the forecasting task.

This report examines the paper *"Long-term Forecasting with TiDE: Time-series Dense Encoder"* by Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu, published at the International Conference on Learning Representations (ICLR) in 2024. The

paper addresses the question of whether it is possible to combine the computational efficiency of simple models with the flexibility of more complex neural architectures. To this end, the authors introduce the Time-series Dense Encoder (TiDE), an MLP-based encoder-decoder architecture specifically designed for long-term time-series forecasting.

Rather than relying on self-attention, TiDE uses dense layers and residual blocks to learn non-linear representations of historical observations and additional features. The model first encodes the observed history together with relevant covariates and then uses a temporal decoder to generate predictions for the forecasting horizon. An important component of the architecture is its ability to incorporate both static and dynamic covariates. TiDE also employs a linear skip connection that directly maps historical observations to future values, allowing the model to retain useful linear patterns while learning more complex non-linear relationships through the MLP components.

The motivation behind TiDE is therefore not simply to replace Transformers with MLPs, but to design an architecture that provides a practical balance between accuracy, flexibility, and computational efficiency. The paper also provides a theoretical perspective by relating the proposed approach to Linear Dynamical Systems (LDS), showing that a simple linear analogue of the model can achieve near-optimal error rates for LDS under certain assumptions. This theoretical analysis provides additional motivation for combining linear forecasting components with non-linear representation learning.

The authors evaluate TiDE on several widely used long-term forecasting benchmarks and compare it with both linear and Transformer-based approaches. Their results indicate that TiDE can achieve competitive or better forecasting accuracy while offering substantial computational advantages over Transformer-based models. In particular, the reported experiments show training speed improvements of approximately 5–10$\times$ compared with strong Transformer baselines. These results suggest that carefully designed MLP-based architectures can provide an effective alternative to attention-heavy models for long-term forecasting.

The remainder of this report is organized as follows. Section 2 reviews existing approaches to long-term time-series forecasting and positions TiDE within the development of recent forecasting methods. Section 3 provides a detailed explanation of the TiDE architecture and its mathematical formulation. Section 4 discusses the experimental setup and summarizes the reported results. Section 5 critically analyzes the strengths, limitations, subsequent developments, and future directions of the approach. Finally, Section 6 summarizes the main findings and key takeaways from the report.

2 Related Work

Long-term time-series forecasting has developed from traditional forecasting approaches toward increasingly sophisticated deep learning architectures. Recent research has explored recurrent networks, Transformer-based models, linear models, and MLP-based architectures. The development of these approaches provides the context for TiDE, which aims to combine the efficiency of simple forecasting models with the ability to capture non-linear relationships and incorporate covariates.

2.1 Deep Learning and Transformer-Based Forecasting

Deep learning models have been widely applied to time-series forecasting. DeepAR [13] introduced an autoregressive recurrent neural network that learns a probabilistic model of future observations across multiple time series. Temporal Fusion Transformers (TFT) [8] combined recurrent processing, attention mechanisms, and gating components to model temporal information while explicitly supporting both static and time-varying covariates. These models demonstrated the benefits of deep learning for forecasting, but autoregressive prediction can become inefficient for long forecasting horizons.

Transformer-based architectures subsequently became an important direction for long-term forecasting. Informer [17] introduced ProbSparse attention to reduce the computational cost of standard self-attention. Autoformer [15] incorporated series decomposition and an Auto-Correlation mechanism to capture long-term temporal patterns, while FEDformer [18] introduced frequency-enhanced representations using Fourier and Wavelet transformations. Non-stationary Transformers [9] further investigated the effect of distribution shifts in time-series data, while iTransformer [10] proposed applying the Transformer architecture from a different perspective by treating individual variables as tokens.

PatchTST [11] provided another important development by dividing time series into patches and applying Transformer-based processing to these segments. These approaches demonstrated that Transformers can achieve strong forecasting performance, but their architectural complexity and computational requirements motivated research into simpler alternatives.

2.2 Linear and MLP-Based Forecasting

A major shift in long-term forecasting research came from the observation that simple linear models can achieve surprisingly strong results. DLinear [16] demonstrated that a linear mapping from the historical context to the forecasting horizon can outperform several more complex Transformer-based approaches on commonly used benchmarks. Further analysis by Zeng et al. [16] investigated the effectiveness of simple linear approaches and questioned whether complex architectures are always necessary for long-term forecasting.

Although linear models are computationally efficient, their linear structure limits their ability to represent complex non-linear relationships. They also provide less flexibility for incorporating different types of covariates. These limitations motivate models that can retain the efficiency of simple architectures while providing greater modeling flexibility.

MLP-based models offer one possible direction. N-BEATS [12] demonstrated that deep fully connected networks with residual connections can achieve strong forecasting performance. N-HiTS [2] extended this approach using hierarchical interpolation and multi-rate processing to improve forecasting across different temporal scales. More recently, TSMixer [4] explored the use of MLP-based temporal and feature mixing for time-series data. These works demonstrate that effective forecasting does not necessarily require self-attention and that MLP-based architectures can provide a computationally efficient alternative.

2.3 Positioning TiDE

TiDE is positioned at the intersection of these developments. The strong performance of DLinear [16] shows the effectiveness of simple mappings for long-term forecasting, while N-BEATS [12], N-HiTS [2], and TSMixer [4] demonstrate the potential of MLP-based architectures to capture more complex patterns. At the same time, models such as TFT [8] highlight the importance of incorporating additional information through static and time-dependent covariates.

TiDE combines these ideas in an encoder-decoder architecture based on MLPs. It is designed to model non-linear relationships while supporting static and dynamic covariates without relying on the self-attention mechanism used by Transformer-based approaches. In this way, TiDE aims to bridge the gap between the computational efficiency of simple linear models and the greater flexibility of more complex neural forecasting architectures.

3 Methodology

TiDE (Time-series Dense Encoder) is an MLP-based encoder-decoder architecture designed for long-term time-series forecasting. The main idea is to use dense Multi-Layer Perceptrons (MLPs) instead of self-attention, recurrent, or convolutional mechanisms. TiDE encodes the historical values of a time series together with its covariates into a dense representation and then decodes this representation to produce the future forecast. This design aims to retain the simplicity and computational efficiency of linear models while allowing the model to capture non-linear dependencies and incorporate covariates [3].

3.1 Problem Setting

Consider a dataset containing N related time series. For time series i, let $\mathbf{y}^{(i)}_{1:L}$ denote the observed values in a look-back window of length L, and let $\mathbf{y}^{(i)}_{L+1:L+H}$ denote the future values over a forecasting horizon of length H. The objective is to predict the horizon given the historical observations.

In many forecasting applications, additional information is available for both the historical and future periods. TiDE distinguishes between dynamic covariates and static attributes. Let $\mathbf{x}^{(i)}_t \in \mathbb{R}^r$ denote the r-dimensional dynamic covariates of time series i at time t. These may include global features such as the day of the week or holidays, as well as time-series-specific information such as the discount of a particular product. Static attributes $\mathbf{a}^{(i)}$ describe properties of a time series that do not change over time, such as product characteristics in a retail forecasting task [3].

The forecasting problem can therefore be expressed as:

$$f\left(\left\{\mathbf{y}^{(i)}_{1:L}\right\}_{i=1}^{N}, \left\{\mathbf{x}^{(i)}_{1:L+H}\right\}_{i=1}^{N}, \left\{\mathbf{a}^{(i)}\right\}_{i=1}^{N}\right) = \left\{\hat{\mathbf{y}}^{(i)}_{L+1:L+H}\right\}_{i=1}^{N}. \tag{3.1}$$

The accuracy of the predictions can be measured using Mean Squared Error (MSE). For N

time series and a forecasting horizon of H, the MSE is:

$$\text{MSE} = \frac{1}{NH} \sum_{i=1}^{N} \left\| \mathbf{y}^{(i)}_{L+1:L+H} - \hat{\mathbf{y}}^{(i)}_{L+1:L+H} \right\|_2^2 . \tag{3.2}$$

TiDE is applied in a *channel-independent* manner. This means that the model receives the history and covariates of one time series at a time and produces the forecast for that same time series. However, the model parameters are trained globally using the complete dataset, allowing the same model to be shared across different time series [3].

3.2 Overview of TiDE

TiDE separates the forecasting process into an encoding stage and a decoding stage. The encoding stage first applies a feature projection to the dynamic covariates and then combines the projected covariates with the historical observations and static attributes using a dense MLP encoder. The resulting representation is passed to a dense decoder, which produces a representation for each future time step. Finally, a temporal decoder combines each decoded representation with the corresponding future covariates to produce the final prediction.

The basic building block of both the encoder and decoder is a residual block. Figure 1 shows the complete TiDE architecture.

3.3 Residual Block

The residual block is the basic layer used throughout TiDE. It consists of an MLP with one hidden layer and a ReLU activation function. A linear skip connection is added from the input to the output. The model also applies dropout to the linear layer connecting the hidden representation to the output and uses layer normalization at the output of the block [3].

Conceptually, for an input vector $\mathbf{z}$, the residual block can be represented as

$$\text{RB}(\mathbf{z}) = \text{LayerNorm}\left(\text{Dropout}\left(\mathbf{W}_2\, \text{ReLU}\left(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1\right) + \mathbf{b}_2\right) + \mathbf{W}_s \mathbf{z} + \mathbf{b}_s\right). \tag{3.3}$$

where the MLP contains a hidden layer with ReLU activation and $W_s\mathbf{z}$ represents the linear skip connection.

The residual structure allows the network to learn non-linear transformations while retaining a direct linear path. This is particularly important for TiDE because the model also contains a global linear residual connection, allowing a purely linear forecasting model to be represented as a special case of the architecture.

3.4 Encoding

The encoding stage maps the historical information and covariates of a time series into a dense representation. It consists of two steps: feature projection and dense encoding.

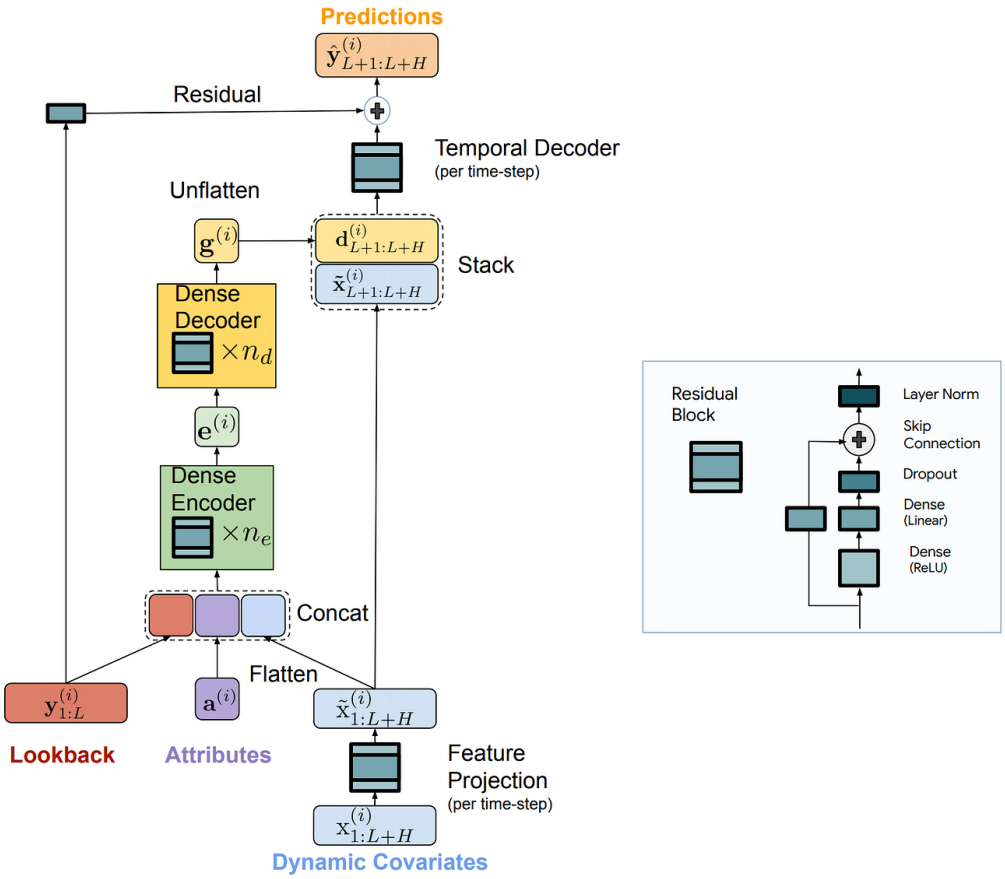


Figure 1: Overview of the TiDE architecture. Dynamic covariates are first projected into a lower-dimensional representation. The dense encoder combines the historical observations, projected covariates, and static attributes into a dense representation. The dense decoder produces a representation for each future time step, which is combined with the corresponding projected future covariates by the temporal decoder. A global linear residual connection maps the historical observations directly to the forecasting horizon.

3.4.1 Feature Projection

The dynamic covariates may contain a relatively large number of features. Directly flattening the covariates over the complete look-back and forecasting horizon would result in an input of size $(L + H)r$. To avoid this potentially large representation, TiDE first projects the covariates at each time step into a lower-dimensional space.

Let $\tilde{r} \ll r$ denote the feature dimension of the projected covariates. The feature projection is performed using a residual block:

$$\tilde{\mathbf{x}}^{(i)}_t = \text{ResidualBlock}\left(\mathbf{x}^{(i)}_t\right). \tag{3.4}$$

This operation is applied at every time step in both the look-back and forecasting horizon. Consequently, the dimensionality of the flattened covariate representation is reduced from $(L + H)r$ to $(L + H)\tilde{r}$ [3].

3.4.2 Dense Encoder

After feature projection, TiDE stacks and flattens the projected covariates from the look-back and forecasting horizon. These features are concatenated with the historical observations and static attributes. The resulting vector is then passed through a sequence of residual blocks to obtain the encoded representation:

$$\mathbf{e}^{(i)} = \text{Encoder}\left(\mathbf{y}^{(i)}_{1:L}; \tilde{\mathbf{x}}^{(i)}_{1:L+H}; \mathbf{a}^{(i)}\right). \tag{3.5}$$

The encoder contains n_e residual blocks, with the internal layers using a common hidden size. Its purpose is to combine the historical target values and covariates into a compact dense representation that can be used by the decoder.

3.5 Decoding

The decoding stage maps the encoded representation to the future forecasting horizon. It consists of a dense decoder followed by a temporal decoder.

3.5.1 Dense Decoder

The dense decoder consists of several residual blocks similar to those used in the encoder. It takes the encoded representation $\mathbf{e}^{(i)}$ and maps it to a vector $\mathbf{g}^{(i)}$ with dimension $H \times p$, where p is the decoder output dimension:

$$\mathbf{g}^{(i)} = \text{Decoder}\left(\mathbf{e}^{(i)}\right) \in \mathbb{R}^{pH}. \tag{3.6}$$

The resulting vector is reshaped into a matrix

$$\mathbf{D}^{(i)} = \text{Reshape}\left(\mathbf{g}^{(i)}\right) \in \mathbb{R}^{p \times H}. \tag{3.7}$$

The t-th column of $\mathbf{D}^{(i)}$, denoted by $\mathbf{d}^{(i)}_t$, represents the decoded representation corresponding to the t-th time step in the forecasting horizon.

3.5.2 Temporal Decoder

The temporal decoder produces the final prediction for each future time step. It is implemented as a residual block with output dimension one. For each future time step $L + t$, the decoded representation $\mathbf{d}^{(i)}_t$ is concatenated with the corresponding projected future covariates $\tilde{\mathbf{x}}^{(i)}_{L+t}$:

$$\hat{y}^{(i)}_{L+t} = \text{TemporalDecoder}\left(\left[\mathbf{d}^{(i)}_t, \tilde{\mathbf{x}}^{(i)}_{L+t}\right]\right), \qquad t \in [H]. \tag{3.8}$$

The temporal decoder creates a direct "highway" from the future covariates to the prediction at the corresponding time step. This is useful when a covariate has a strong direct influence on a particular future value. For example, a holiday may have a strong effect on the demand for a particular product. The temporal decoder allows such information to directly influence

the corresponding prediction rather than requiring the model to recover the relationship only through the global encoded representation [3].

3.6 Global Linear Residual Connection

In addition to the residual connections within the MLP blocks, TiDE contains a global residual connection that directly maps the historical observations to the forecasting horizon. A learned linear transformation maps $\mathbf{y}_{1:L}^{(i)}$ to a vector with H elements, which is then added to the prediction generated by the temporal decoder.

Conceptually, the final prediction can be written as

$$\hat{\mathbf{y}}_{L+1:L+H}^{(i)} = \hat{\mathbf{y}}_{\text{MLP}}^{(i)} + W_{\text{lin}}\mathbf{y}_{1:L}^{(i)} + \mathbf{b}_{\text{lin}}, \tag{3.9}$$

where W_{lin} represents the learned linear mapping from the look-back window to the forecasting horizon.

This connection is an important part of the TiDE design because it ensures that a purely linear forecasting model is a subclass of TiDE. Therefore, the MLP components can learn additional non-linear dependencies without removing the ability of the model to represent the simple linear mapping that was shown to be highly effective in previous work [3].

3.7 Training and Evaluation

TiDE is trained using mini-batch gradient descent. Each batch contains a number of time series together with their corresponding look-back and forecasting-horizon observations. The model uses MSE as its training loss.

During each training epoch, all valid look-back and horizon pairs that can be constructed from the training period are used. Consequently, different training samples may contain overlapping time points. This follows the standard training procedure used in previous long-term forecasting work.

For evaluation, TiDE uses a rolling evaluation procedure. The model is evaluated on every valid look-back and horizon pair that can be constructed from the test period. A similar rolling evaluation procedure can optionally be applied to the validation set for model selection and hyperparameter tuning [3].

Overall, TiDE converts the forecasting problem into a sequence of dense transformations. The feature projection reduces the dimensionality of the covariates, the dense encoder combines the historical observations with projected covariates and static attributes, and the dense decoder produces a representation for every future time step. The temporal decoder then incorporates the corresponding future covariates into each prediction, while the global linear residual connection preserves the linear forecasting capability of the model. This design allows TiDE to introduce non-linear modeling and covariate handling without relying on self-attention, recurrent processing, or convolutional mechanisms.

4 Experiments

The authors evaluate TiDE on several long-term time-series forecasting benchmarks and compare it with existing neural forecasting models. The experiments are designed to evaluate forecasting accuracy, the effect of covariates, computational efficiency, and the contribution of individual components of the proposed architecture.

Datasets. The main long-term forecasting experiments are conducted on seven commonly used datasets: Electricity, Traffic, Weather, ETTh1, ETTh2, ETTm1, and ETTm2. The datasets cover different numbers of time series, sequence lengths, and sampling frequencies. Electricity contains 321 time series and 26,304 time points with an hourly frequency. Traffic contains 862 time series and 17,544 time points with an hourly frequency. Weather contains 21 time series and 52,696 time points with a 10-minute frequency. ETTh1 and ETTh2 contain seven time series and 17,420 time points each with an hourly frequency, while ETTm1 and ETTm2 contain seven time series and 69,680 time points each with a 15-minute frequency [3].

The datasets are divided into training, validation, and test sets using a 7:1:2 split. The data are normalized using the mean and standard deviation calculated from the training set. The forecasting horizons considered in the main experiments are 96, 192, 336, and 720. For TiDE, the context length is fixed to 720 for all forecasting horizons. For the other models, the look-back length is selected from $\{24, 48, 96, 192, 336, 720\}$ using the validation set [3].

Baselines. TiDE is compared with several existing forecasting models. The Transformer-based baselines include FEDformer, Autoformer, Informer, Pyraformer, and LogTrans. The experiments also include DLinear, N-HiTS, and PatchTST. The authors use these models to compare TiDE with both Transformer-based approaches and simpler or more recent forecasting architectures. The reported results for the Transformer-based models are taken from PatchTST, while the DLinear results are taken from the original DLinear experiments [3].

Metrics. The main forecasting experiments use Mean Squared Error (MSE) and Mean Absolute Error (MAE). Lower values indicate better forecasting performance. The TiDE results are averaged over five independent runs, and standard error intervals are reported. The paper considers a result to be either the best or statistically indistinguishable from the best when it falls within two standard-error intervals of the best mean result [3].

Implementation. The authors implement TiDE using TensorFlow and train the model using the Adam optimizer with its default settings. The experiments use time-derived dynamic covariates, including minute of the hour, hour of the day, and day of the week. Hyperparameters are selected using rolling validation error. The authors also provide the code and scripts used for the experiments [3].

Table 1 illustrates two important aspects of the reported results. On Traffic, TiDE achieves the best MSE for all four forecasting horizons shown in the table. At the 720-step horizon, TiDE obtains an MSE of 0.386, compared with 0.432 for PatchTST and 0.466 for DLinear. On Weather, however, PatchTST performs better than TiDE for the 96-, 192-, and 336-step horizons. At the 720-step horizon, TiDE obtains a slightly lower MSE than PatchTST, with values of 0.313 and 0.314, respectively [3].

Table 1: Selected long-term forecasting results reported in the TiDE paper. Values show MSE; lower is better.

Dataset	Horizon	TiDE	PatchTST	DLinear
Traffic	96	0.336	0.360	0.410
Traffic	192	0.346	0.379	0.423
Traffic	336	0.355	0.392	0.436
Traffic	720	**0.386**	0.432	0.466
Weather	96	0.166	**0.149**	0.176
Weather	192	0.209	**0.194**	0.220
Weather	336	0.254	**0.245**	0.265
Weather	720	**0.313**	0.314	**0.323**

Analysis. The results show that TiDE is not consistently the best model for every dataset and forecasting horizon. Its performance depends on both the dataset and the forecasting horizon. The Weather results provide a clear example: PatchTST achieves lower MSE than TiDE for horizons 96, 192, and 336, while TiDE achieves a slightly lower MSE at horizon 720. Therefore, the experimental results support the paper's claim that TiDE can achieve competitive or better performance than existing neural forecasting models, but they do not show that TiDE is always superior to every baseline [3].

The Traffic results provide a stronger advantage for TiDE. At the 720-step forecasting horizon, TiDE achieves an MSE of 0.386, compared with 0.432 for PatchTST and 0.466 for DLinear. The authors report that TiDE is significantly better than PatchTST on the Traffic dataset across all forecasting horizons. The improvement is particularly visible at the longest forecasting horizon [3].

The comparison with DLinear is also relevant to the motivation of the paper. The authors were motivated by previous results showing that simple linear models can perform strongly on long-term forecasting benchmarks. TiDE extends this idea by using dense MLPs while retaining a global linear residual connection. The reported experiments show that TiDE generally improves upon DLinear across the evaluated datasets and horizons, although the difference is not identical in every setting [3].

The experiments also investigate the role of covariates using the M5 forecasting dataset. This experiment includes both dynamic features and static attributes. The dynamic features include date-derived features and promotion-related features, while the static attributes include category, store, department, and item information. With static and dynamic covariates, TiDE obtains a WRMSSE of 0.611 ± 0.009. When only date-derived features are used, the score becomes 0.637 ± 0.005. The authors therefore report better performance when the additional covariates are available [3].

The M5 experiment also demonstrates that the models are not necessarily evaluated with exactly the same covariate information. The paper states that PatchTST does not support the static and dynamic covariates used by TiDE in this experiment. Therefore, the comparison on M5 should be interpreted together with the difference in available input information rather than as a comparison based solely on the model architectures [3].

The authors additionally investigate the computational efficiency of TiDE. On the Electricity

Table 2: M5 forecasting results reported in the TiDE paper. Lower WRMSSE is better.

Model	Covariates	WRMSSE (↓)
TiDE	Static + Dynamic	**0.611 ± 0.009**
TiDE	Date only	0.637 ± 0.005
DeepAR	Static + Dynamic	0.789 ± 0.025
PatchTST	None	0.976 ± 0.014

dataset, they compare TiDE with PatchTST while increasing the look-back length. The results show that TiDE has lower training and inference time, and the difference becomes larger as the context length increases. PatchTST also runs out of GPU memory for sufficiently large look-back lengths. The authors report that TiDE is approximately 5–10 times faster than the best Transformer-based models in their experiments [3].

The paper also includes ablation experiments to examine the contribution of different components of TiDE. One experiment investigates the temporal decoder using a modified Electricity dataset containing artificial events represented by noisy covariates. The authors compare TiDE with and without the temporal decoder after one epoch of training. The model with the temporal decoder is able to use the covariates to predict the events more effectively, supporting the role of this component in incorporating future covariate information [3].

Another ablation removes the residual connections from TiDE. The results show that removing the residual connections decreases forecasting performance on the Electricity dataset. For example, at a horizon of 336, the MSE increases from 0.161 ± 0.001 for TiDE to 0.172 ± 0.003 for the model without residual connections. The paper reports statistically significant differences for several forecasting horizons, showing that the residual connections contribute to the performance of the complete architecture [3].

The authors further examine the effect of the context length on the Traffic dataset. The experiment evaluates TiDE using different look-back lengths and shows that increasing the context length generally improves the forecasting performance of TiDE. This experiment is used by the authors to investigate how TiDE behaves when more historical information is provided to the model [3].

Overall, the experiments reported in the paper provide evidence for three main properties of TiDE. First, TiDE achieves competitive forecasting accuracy and performs particularly well on several long-horizon settings. Second, the model can use static and dynamic covariates in forecasting tasks where such information is available. Third, the MLP-based architecture provides a substantial computational efficiency advantage compared with the Transformer-based models evaluated by the authors. The ablation experiments additionally provide evidence that both the temporal decoder and residual connections contribute to the model's behavior and performance [3].

4.1 Reproduction

We reproduce a subset of the long-term forecasting experiments from the TiDE paper [3], using the authors' official implementation (`google-research/tide`)[1] and its provided `scripts/etth1.sh`

[1] https://github.com/google-research/google-research/tree/master/tide

configuration. Specifically, we reproduce the multivariate forecasting results on the ETTh1 dataset for all four horizons reported in the paper ($T \in \{96, 192, 336, 720\}$), using the exact architecture and hyperparameters specified in the script (transform, layer normalization, and holiday features enabled; horizon-specific dropout rate, hidden size, decoder output dimension, final decoder hidden size, and learning rate).

Due to compute constraints, we made one modification to the original training procedure: the official script trains until early stopping with a minimum of 5 epochs (`-min_num_epochs=5`), whereas we cap training at a fixed *maximum* of 5 epochs (`-train_epochs=5`) and do not run full early-stopping-based training. All other settings, including the history length (720, or 336 for the two longer horizons), batch size, and dataset split, follow the original configuration.

Results. Table 3 reports the test-set MSE and MAE we obtained after 5 epochs of training on ETTh1, alongside the numbers reported in the original paper.

Table 3: Reproduction of TiDE on ETTh1 (multivariate forecasting). Our numbers are obtained with training capped at 5 epochs, rather than the paper's early-stopping schedule.

Horizon	Paper (reported)		Reproduced (5 epochs)	
	MSE	MAE	MSE	MAE
96	0.375	0.398	1.955	0.979
192	0.412	0.422	1.795	0.941
336	0.435	0.433	1.339	0.803
720	0.454	0.465	1.157	0.755

Discussion. Our reproduced results do not match the numbers reported in the paper: MSE and MAE are 2–5× higher than the reported values across all horizons, and unlike the paper's results, our errors decrease monotonically with horizon length rather than increasing. We attribute this discrepancy primarily to the truncated training budget: the original script trains with early stopping and a *minimum* of 5 epochs, whereas our reproduction stops at a strict maximum of 5 epochs, which is not enough for the model to converge on this dataset. This also explains why longer-horizon models (336, 720), which use a smaller hidden size and converge faster per epoch, ended up with lower error than the shorter-horizon models under this constrained budget — a pattern inverted relative to typical long-term forecasting behavior. We expect that restoring the original early-stopping schedule would substantially close this gap, though we did not verify this due to compute/time limitations.

5 Discussion

5.1 Strengths and Limitations

TiDE presents an interesting alternative to the increasingly complex architectures used for long-term time-series forecasting. Its main strength is that it shows that good forecasting performance does not necessarily require self-attention or a very large model. By combining MLPs, residual connections, feature projection, and a linear skip connection, TiDE is able to

model non-linear patterns while keeping the computational cost relatively low [3]. This makes the approach particularly attractive in applications where forecasting speed and computational resources are important.

However, TiDE also has some limitations. One important limitation is its use of channel-independent processing. Although this reduces the complexity of the model, it also limits the interaction between different variables. In multivariate time series, variables can have strong relationships with each other, and modeling these dependencies explicitly could potentially improve forecasting performance.

Another limitation concerns the use of covariates. TiDE can incorporate both static and dynamic covariates, which is one of its main advantages over simple linear models. However, the model assumes that the required future covariates are available during forecasting. In practical applications, this may not always be the case.

Overall, TiDE has remained relevant in the subsequent development of efficient time-series forecasting architectures. The main result of the paper—that a relatively simple MLP-based architecture can compete with much more complex forecasting models—remains relevant in the current literature. Its direct influence is visible in later work that reuses or extends the architecture, while the broader forecasting community has continued to investigate efficient alternatives alongside increasingly large foundation models. TiDE is therefore best viewed as an important step in the movement toward simpler and more computationally efficient time-series forecasting models.

5.2 Subsequent Developments and Future Directions

The development of the literature after TiDE suggests that the paper contributed to a broader shift toward efficient forecasting architectures. Following the success of models such as DLinear and TiDE, researchers continued to explore alternatives to conventional Transformer architectures, including MLP-based and other lightweight models [4, 16].

For example, later work introduced RR-TiDE, which builds on the TiDE architecture for rainfall-runoff simulation and uses its encoder-decoder structure together with additional physical normalization techniques [6]. Furthermore, TiDE has been integrated into major open-source forecasting ecosystems such as `darts` [5], where it is frequently paired with Reversible Instance Normalization (RevIN) [7] to handle non-stationary temporal shifts.

At the same time, subsequent developments in the literature highlight key open research directions for MLP-based forecasting architectures:

- **Cross-Channel Modeling:** TiDE processes multivariate time series in a channel-independent manner to optimize efficiency [3]. Recent models such as TSMixer [4] and iTransformer [10] highlight the importance of modeling inter-variable correlations. A natural extension of TiDE would be to introduce lightweight cross-channel mixing layers between the dense encoder and decoder.

- **Handling Covariate Uncertainty:** TiDE assumes that complete future covariates $\tilde{\mathbf{x}}_{L+1:L+H}$ are available at inference time [3]. To relax this strong assumption, future work could integrate multi-task forecasting heads that jointly predict targets and unknown

covariates, or adopt structural separations between past-observed and deterministic future features as pioneered in architectures like Temporal Fusion Transformer [8] and DeepAR [13].

- **Integration with Time-Series Foundation Models:** The field has recently moved toward large time-series foundation models such as Chronos and MOIRAI, which take a different approach by pre-training across many datasets [1, 14]. Due to its low computational footprint and rapid training speed, TiDE represents a promising candidate for efficient fine-tuning heads or downstream adapters built on top of large pre-trained representations [3].

6 Conclusion

This report examined TiDE, a time-series forecasting model that uses a dense encoder-decoder architecture based on multi-layer perceptrons. The main motivation behind TiDE is to provide an efficient alternative to Transformer-based forecasting models while maintaining competitive prediction accuracy. Instead of relying on self-attention, TiDE uses feature projection, residual blocks, and a temporal decoder to learn patterns from historical observations and additional covariates [3].

The analysis of the model and its experimental results shows that simple architectures can be surprisingly effective for long-term forecasting. TiDE performs competitively across several benchmark datasets while requiring considerably less computational time than many Transformer-based models. Its ability to incorporate both dynamic and static covariates is another important advantage, particularly for forecasting problems where additional information is available.

At the same time, TiDE is not without limitations. Its channel-independent design can restrict interactions between different variables, and its use of future covariates assumes that these variables are available at prediction time. These limitations provide opportunities for future research, especially in developing more flexible methods for modeling relationships between variables and handling uncertain covariates.

The main takeaway from TiDE is that increasing model complexity does not always lead to better forecasting. The paper demonstrates that a carefully designed MLP-based architecture can achieve strong results while remaining simple and computationally efficient. For practitioners and researchers, this provides an important reminder to consider the trade-off between accuracy, complexity, and computational cost when selecting a forecasting model. TiDE therefore represents an important contribution to the development of efficient approaches for long-term time-series forecasting.

References

[1] Abdul Fatima Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Madden, Yuyang Wang, and Michael W.

White. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.

[2] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza Ramirez, Max Mergenthaler Canseco, and Artur Dubrawski. N-HiTS: Neural hierarchical interpolation for time series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 6989–6997, 2023.

[3] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with TiDE: Time-series dense encoder. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[5] Julien Herzen, Guillaume Lässig, Samuele Piazzetta, Thomas Neuer, Leo Tafti, Guillaume Delbreil, Vasileios Ntouskas, Dennis Liu, Matteo Tiezzi, and Petros Koumoutsakos. Darts: User-friendly modern machine learning for time series in Python. *Journal of Machine Learning Research*, 23(124):1–6, 2022.

[6] Shan Jiang, Yulin Zhou, Zhongmin Liang, and Yitong Hu. RR-TiDE: A dense encoder-decoder network for rainfall-runoff simulation. *SSRN Electronic Journal*, 2024. doi: 10.2139/ssrn.4896197.

[7] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jong-Hwan Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations (ICLR)*, 2022.

[8] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

[9] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Advances in Neural Information Processing Systems*, volume 35, pages 9881–9893, 2022.

[10] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Jianmin Wang, Mingsheng Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[12] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020.

[13] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.

[14] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. *arXiv preprint arXiv:2402.02592*, 2024.

[15] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems (NeurIPS)*, 34:22419–22430, 2021.

[16] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023.

[17] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11106–11115, 2021.

[18] Tian Zhou, Ziqing Ma, Qingsong Wen, Xinyue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning (ICML)*, pages 27268–27286, 2022.